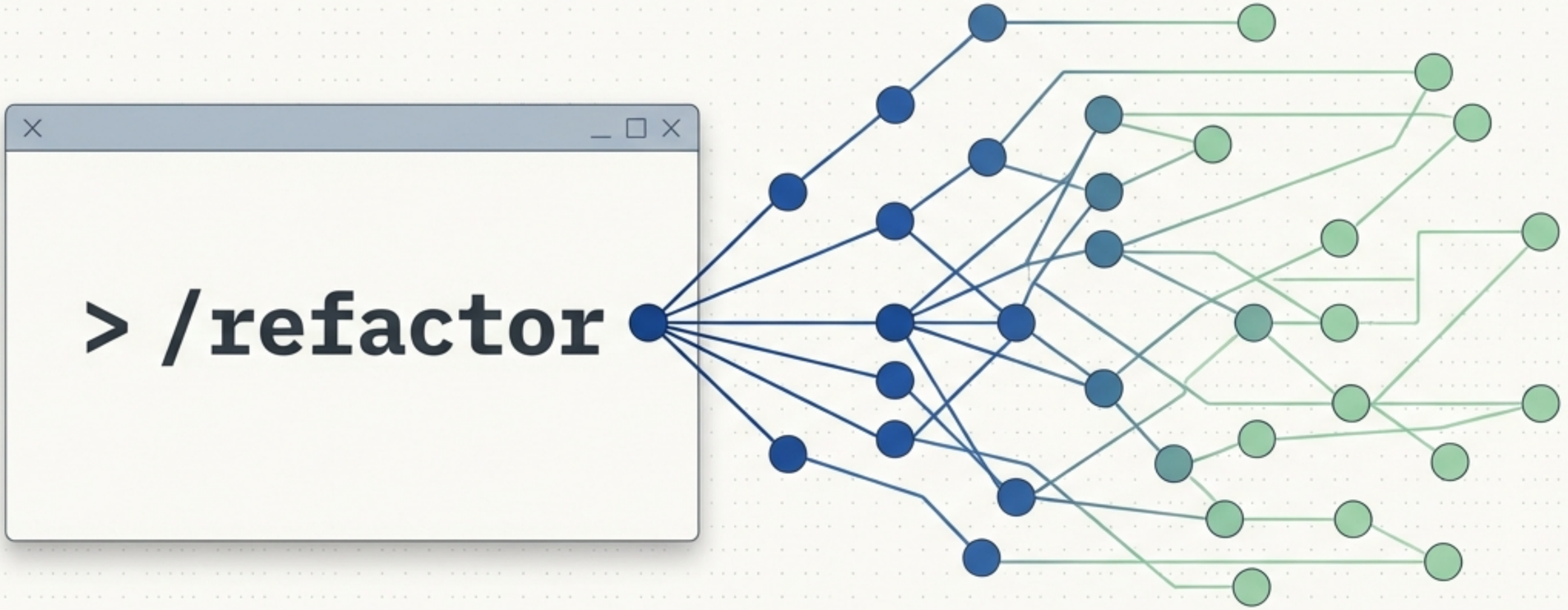
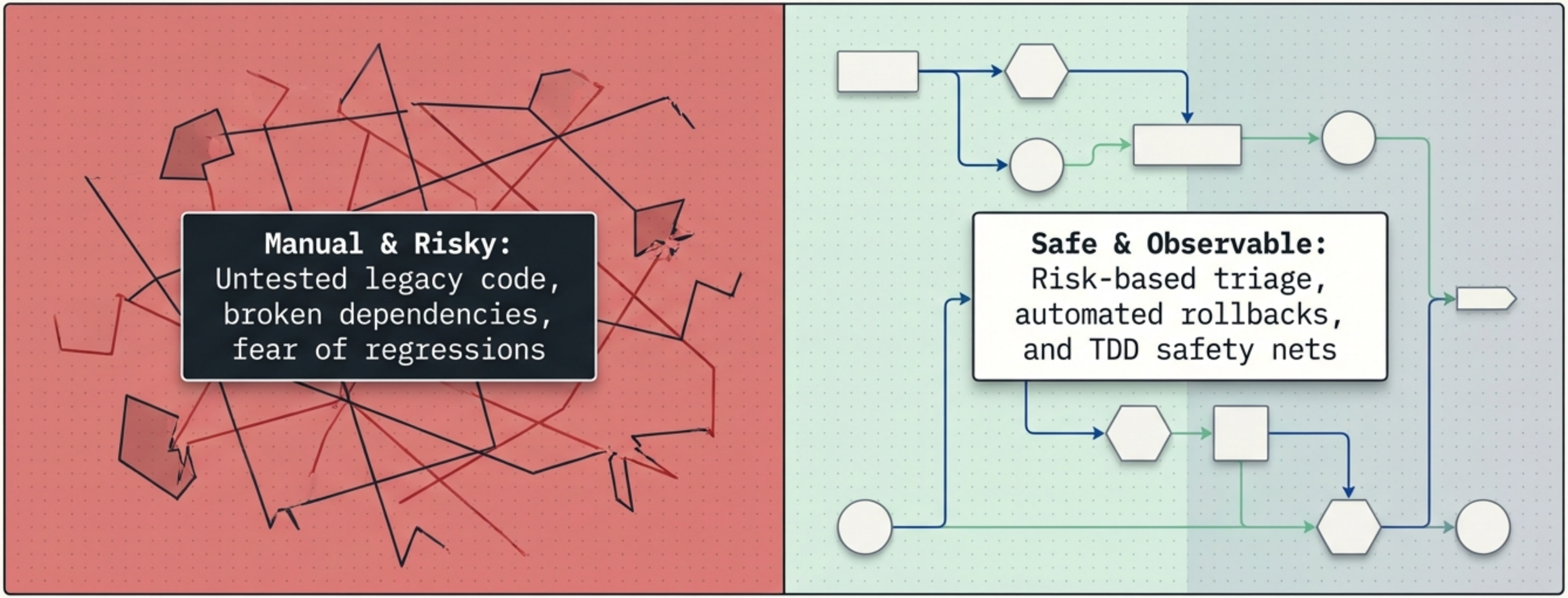


Orchestrator: Claude Code Multi-File Refactoring Engine

Status: Production Ready (Version 1.0.0)

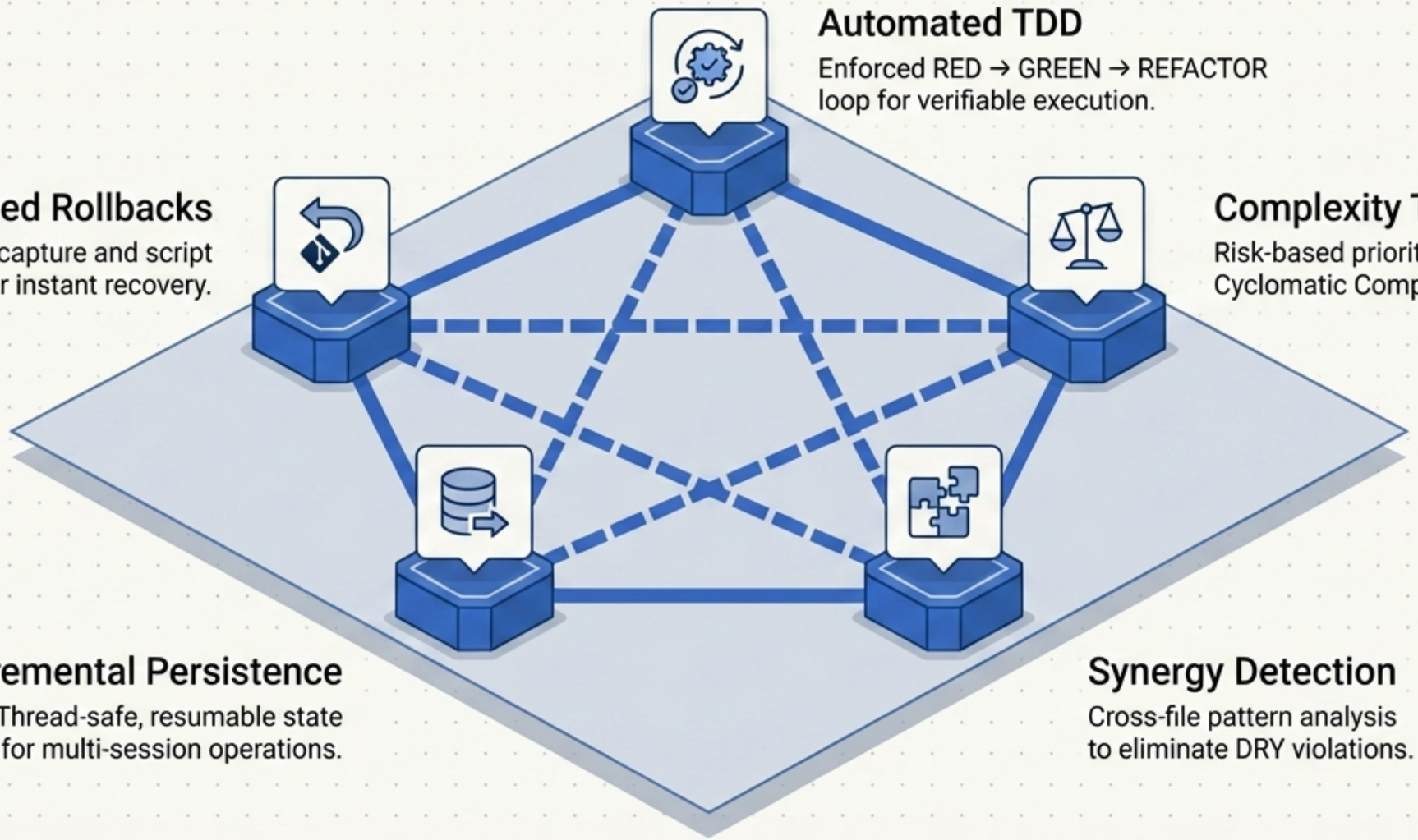
Core Capability: Transforms dense codebases via safe, automated, and observable AI-driven TDD pipelines.

Manual refactoring triggers regressions; automated orchestration guarantees safety



Large-scale refactoring fails when developers lack comprehensive context. The **/refactor** engine eliminates this risk by reading the entire dependency graph, assessing complexity, and securing state before a single file is modified.

Five pillars enforce structural integrity before a single line of code changes.



Automated TDD

Enforced RED → GREEN → REFACTOR loop for verifiable execution.

Git-Based Rollbacks

Automated state capture and script generation for instant recovery.

Complexity Triage

Risk-based prioritization utilizing Cyclomatic Complexity scoring.

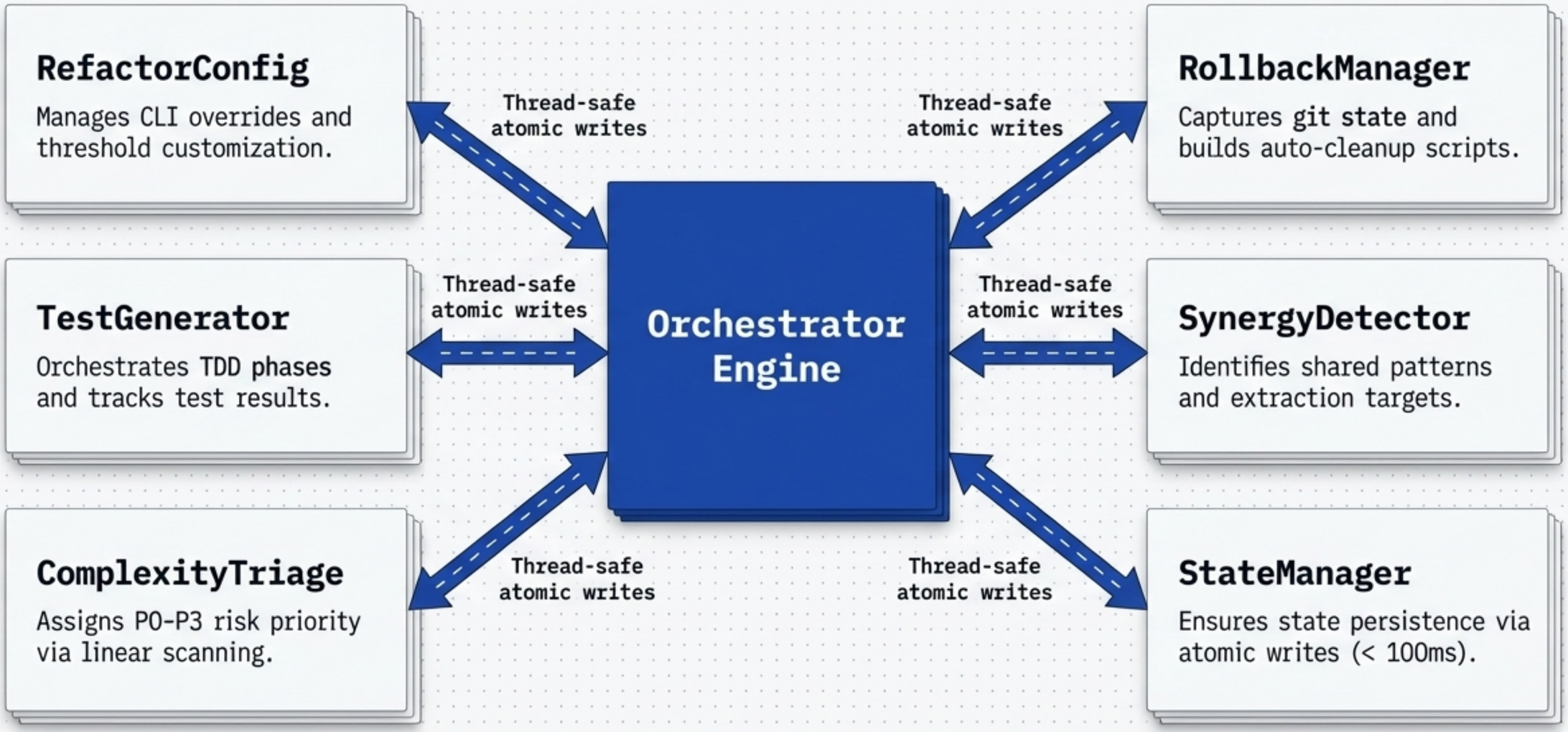
Incremental Persistence

Thread-safe, resumable state capture for multi-session operations.

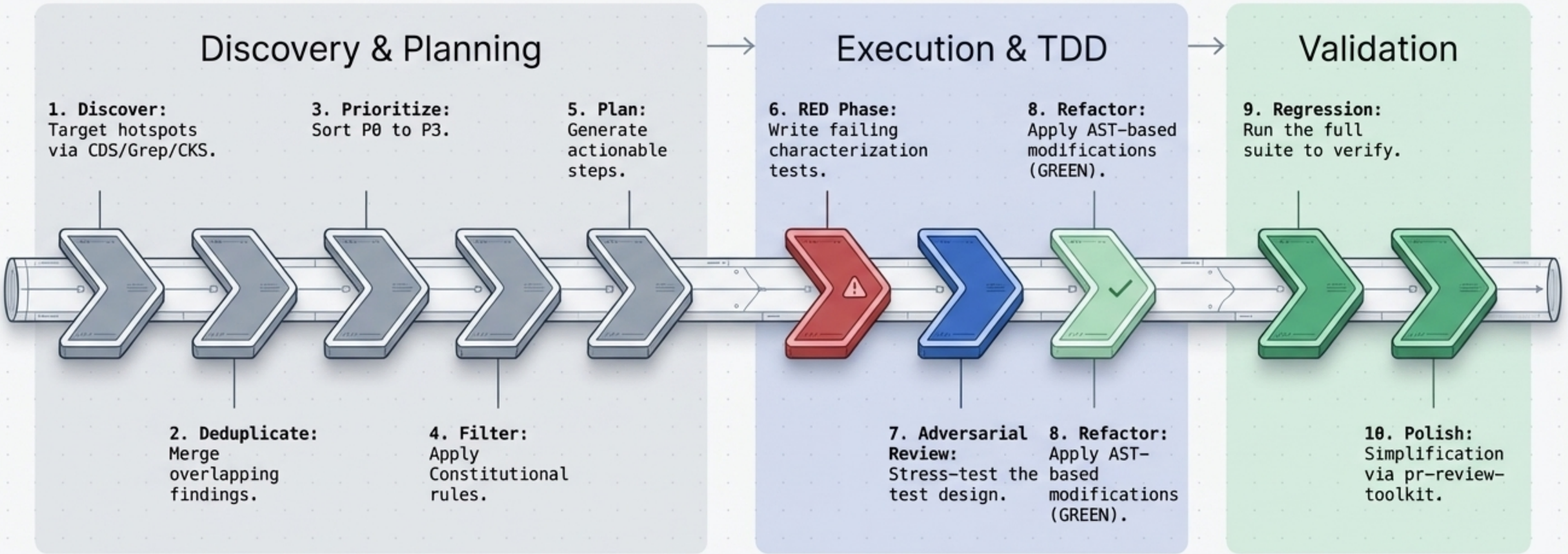
Synergy Detection

Cross-file pattern analysis to eliminate DRY violations.

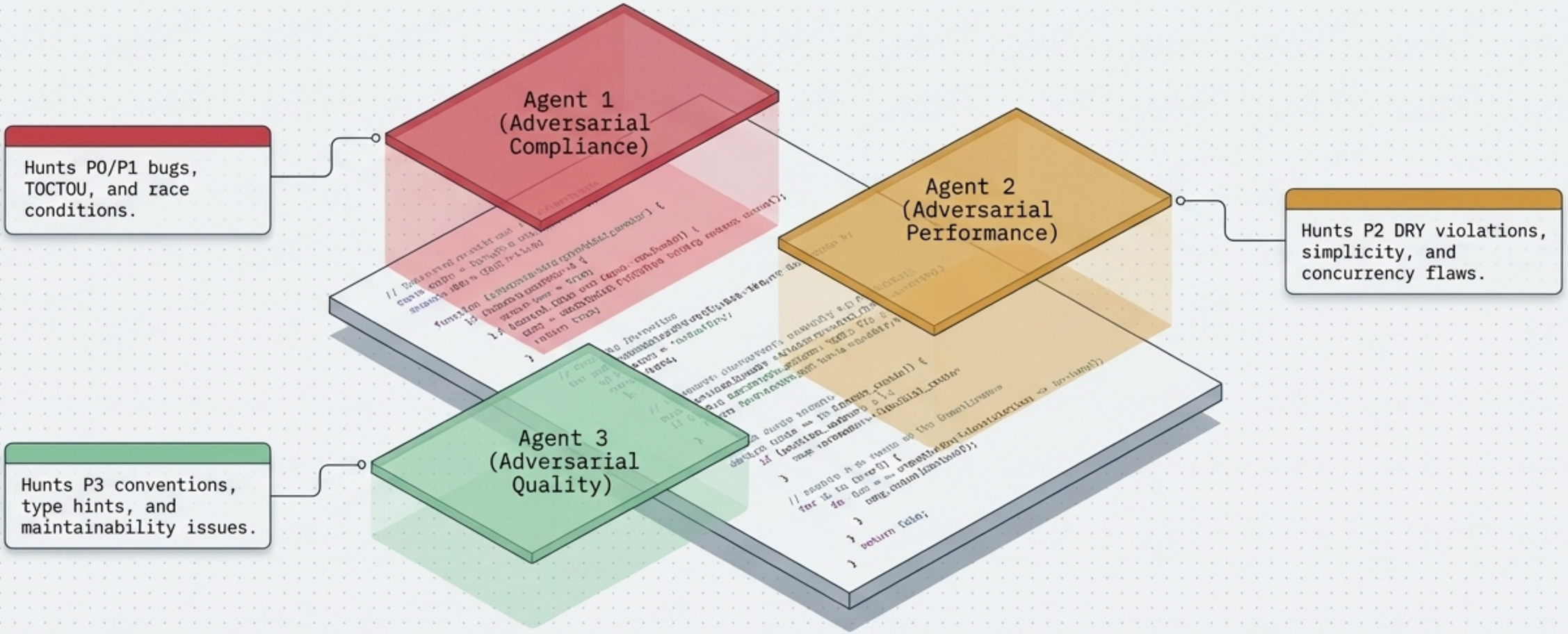
A thread-safe orchestration engine coordinates atomic writes and rollback protection.



Code flows through a sequential surgical pipeline from discovery to regression testing.



Staggered parallel agents hunt for vulnerabilities without context-window bloat



Instead of overwhelming a single prompt, the orchestrator divides the workload across specialized, concurrent personas. Staggered execution ensures maximum codebase coverage while preserving granular attention to detail.

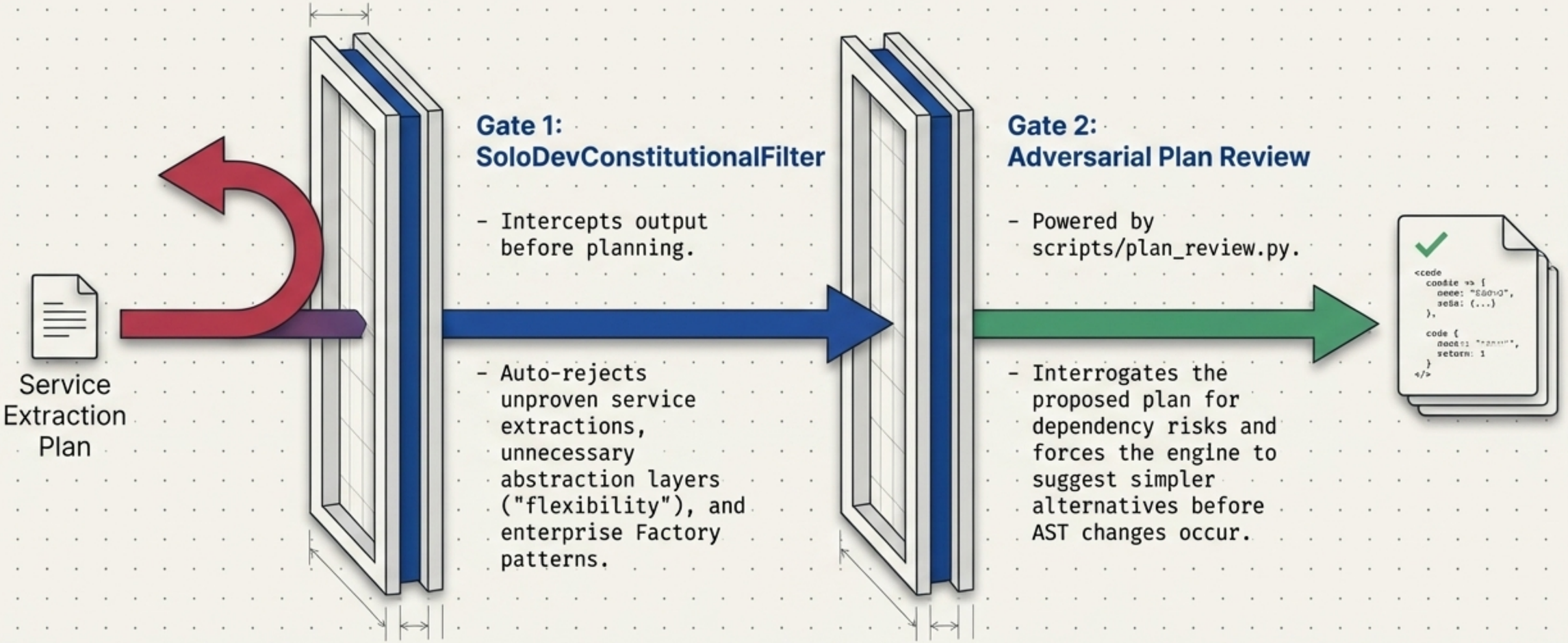
Risk-based complexity triage ensures the most dangerous code is repaired first.

Risk Heatmap Matrix

P0 (Critical Danger)	**CC ≥ 30**	Attacks logic bugs, race conditions, and error states immediately.
P1 (High Complexity)	**CC ≥ 15**	Targets complex error handling and deeply nested logic.
P2 (Medium Complexity)	**CC ≥ 8**	Focuses on DRY violations and structural duplication.
P3 (Low Complexity)	**CC < 8**	Polishes standard conventions and missing type hints.

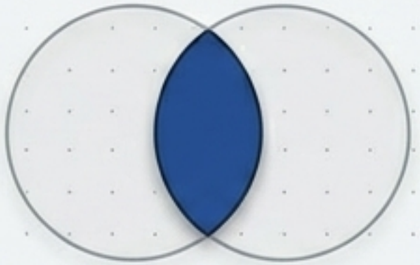
Rule: The engine executes **all priority levels sequentially (P0→P1→P2→P3)** without stopping, **aggressively attacking high-CC hotspots first.**

Defense-in-depth quality gates actively block enterprise bloat and YAGNI patterns



The synergy engine detects cross-file patterns to eliminate hidden duplication.

Extract



Shared logic found in multiple files is isolated to a shared module.

Merge

Combining similar protocol classes or overlapping interfaces.

Consolidate

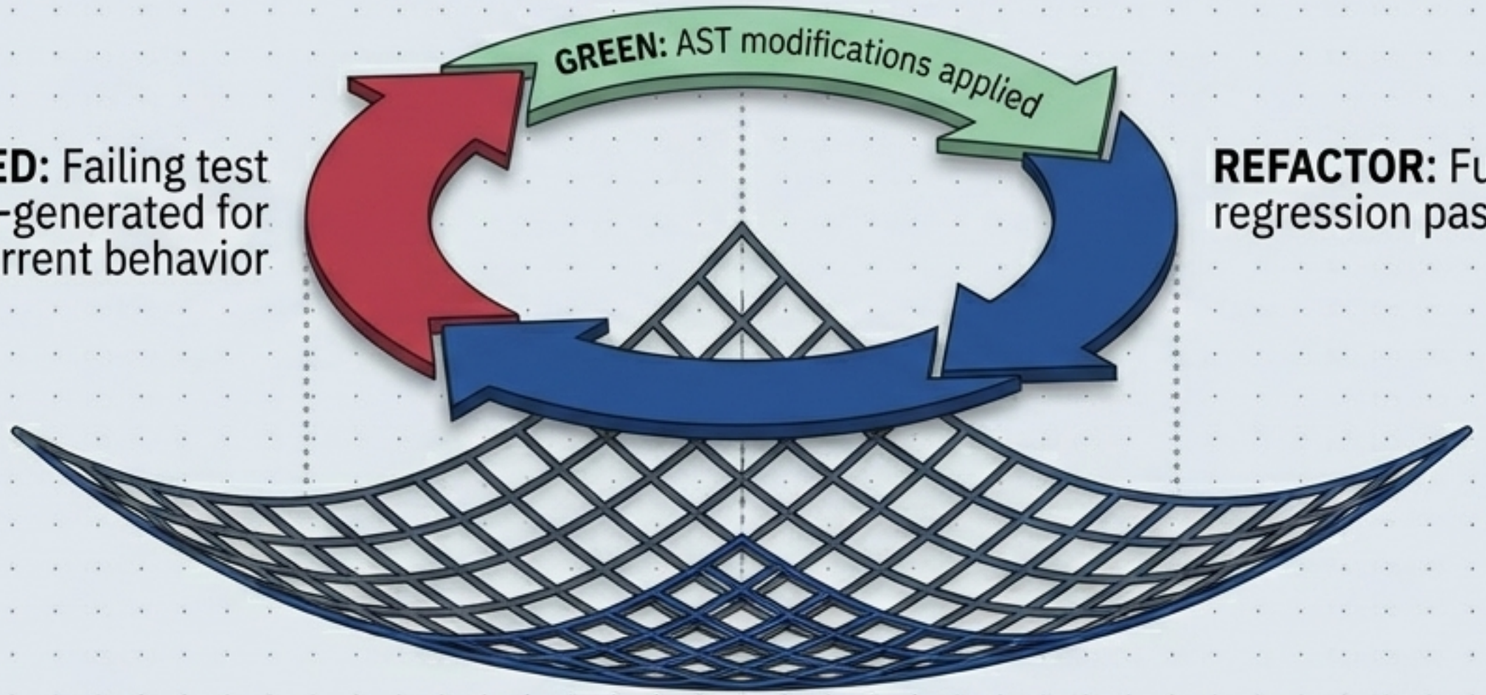
Centralizing scattered configurations or resource access patterns.

Modernize

Updating outdated syntax to modern best practices.

Context7 Integration: Intercepts deprecated APIs (e.g., outdated `requests.Session()`) and actively expands queries to source current Python 2025 best-practice replacements (e.g., `httpx` async).

Mandatory characterization tests and automated rollbacks create an unbreakable safety net.



The RollbackManager Guarantee: Before Phase 1 begins, the engine auto-generates git branches and executable rollback scripts. If the RED phase fails to fail, or the GREEN phase fails to pass, state is instantly restored in under 1 second.

Dynamic focus lenses calibrate the orchestrator's analysis to specific project goals.

--focus security 100% SECURITY

Cranks up Agent 1 targeting injection, auth, and race conditions.

--focus complexity 75% COMPLEXITY

Cranks up Agent 2 targeting high Cyclomatic Complexity and nested logic.

--focus architecture 50% ARCHITECTURE

Balances all agents to evaluate boundary violations and tight coupling.

--focus test 80% TEST

Cranks up Agent 3 targeting missing coverage and test generation.

Three deployment models adapt to active development, testing, and distribution.

☆ Recommended		
SKILLS (Dev Deployment)	**HOOKS** (Dev Testing)	**PLUGINS** (End User)
Target: • Active package developers.	**Target:** • Testing specific `.py` script behaviors.	**Target:** • External user distribution.
Setup: • Edit directly in `packages/refactor/`	**Setup:** • Deployed strictly for hook files in `scripts/hooks/`	**Setup:** • Full package installation via marketplace or GitHub.
Advantage: • Instant feedback. No reinstall required; auto-discovers from `.claude/skills/`	**Advantage:** • Isolated testing environment.	**Advantage:** • Stable, version-locked deployment.

Highly optimized subsystems execute triage, state persistence, and rollbacks in milliseconds.

< 1.0s
Rollback Capture
(Instant Git branch creation)

< 1.0s
Complexity Triage
(Linear AST scanning)

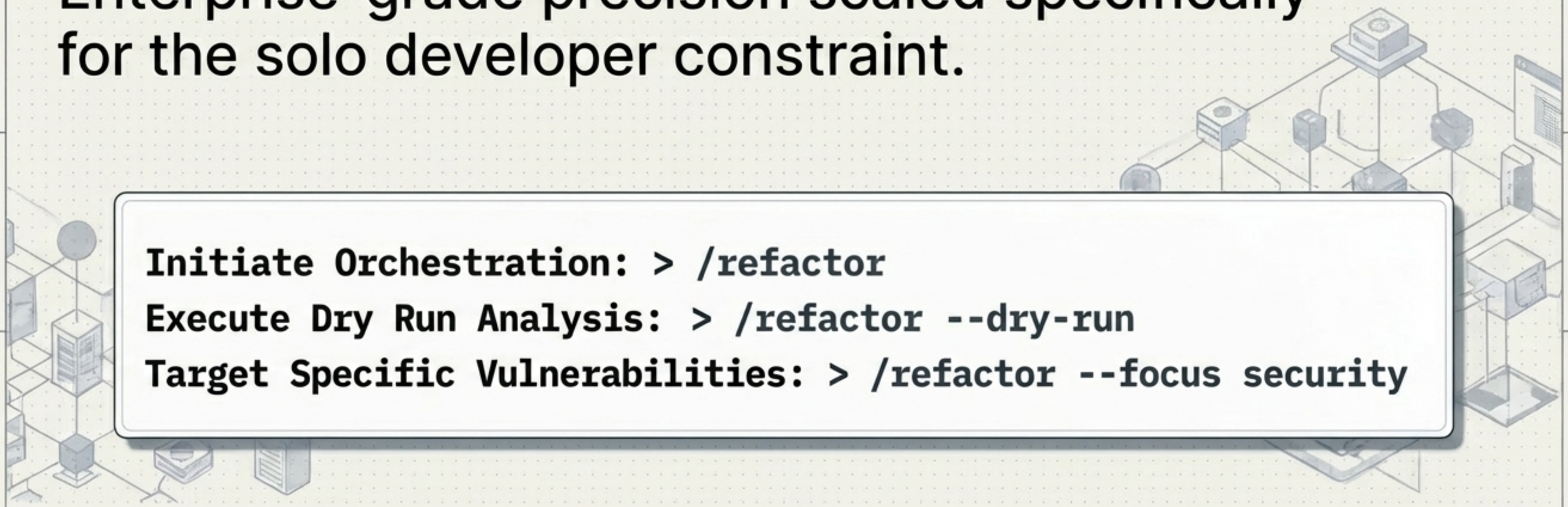
< 100ms
State Persistence
(Thread-safe atomic writes)

2.0s - 10.0s
Synergy Detection
(Scales dynamically with codebase)

Optimization Directives

- For massive codebases, leverage `--cc-threshold` to bypass low-complexity files.
- Restrict synergy detection to specific module directories.
- Enable `incremental` mode to cache progress.

Enterprise-grade precision scaled specifically for the solo developer constraint.



```
Initiate Orchestration: > /refactor
Execute Dry Run Analysis: > /refactor --dry-run
Target Specific Vulnerabilities: > /refactor --focus security
```

The Multi-File Refactoring Orchestrator ensures that code quality scales faster than technical debt, securing legacy systems through automation, evidence collection, and rigorous test validation.